CSI142 Object-Oriented Programming

Methods and Modularity III

Week 3 • Lecture 9

refactoring • method design best practices • bridge to classes
From utility methods to object methods (Week 4)

Fri 13 Feb 2026 • 08:00–09:00 • Room 252-008

Week 3 recap

- Lecture 7: method syntax, return values, pass-by-value, scope
- Lab 3: refactor a menu program into methods, utility method practice
- Lecture 8: overloading, overload selection, shadowing and static context
- Today: put it all together, refactor for clarity, preview objects and classes

Objectives (today)

- Identify responsibilities in a program and map them to methods
- *Refactor code safely:* extract, name, parameterise, test
- Apply method design best practices (small, clear, predictable)
- Use documentation (Javadoc) to communicate method contracts
- See how methods become behaviours of classes in Week 4

Why consolidation and refactoring matter

Without refactoring

- Bug fixes take longer
- Hard to reuse logic
- Hard to read after 2 weeks
- Group work becomes messy

With refactoring

- Code reads like steps
- Each method is testable
- Change happens in one place
- Teams can split work by method

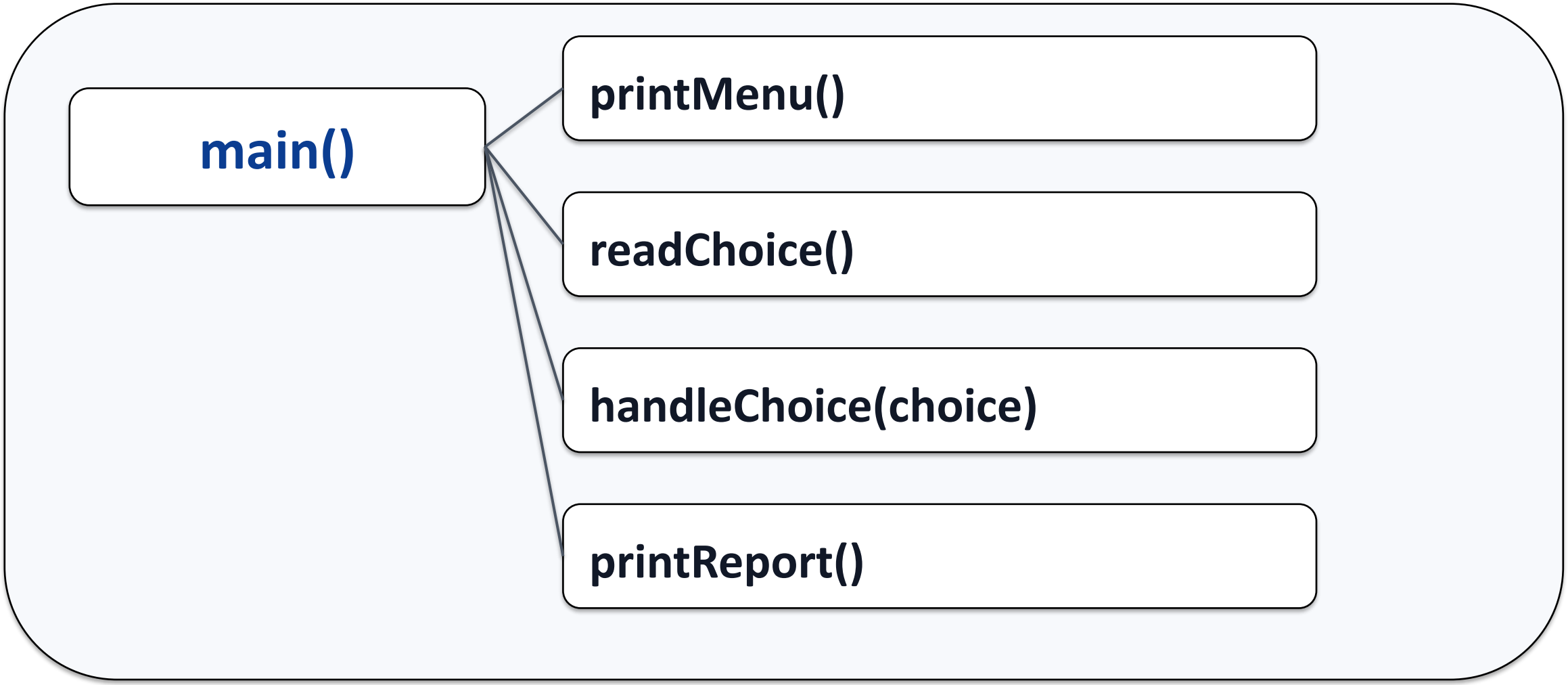
Before refactoring: everything in main()

```
1 public static void main(String[] args) {
2     // lots of code here...
3     // read input, validate, update list, compute average,
print report
4     // all mixed together in one loop
5 }
```

Key points

- Mixed responsibilities;
- Hard to test one part;
- Hard to reuse

After refactoring: main() becomes a coordinator



A practical refactoring recipe

1) Pick one responsibility, extract it into a new method

2) Name the method after what it does (verb + object)

3) Decide parameters: what does the method need to know?

4) Decide return value: what should it produce?

5) Re-run often: small safe steps

Testing small methods (without extra tools)

```
1 static int max(int a, int b) {
2     return (a > b) ? a : b;
3 }
4
5 public static void main(String[] args) {
6     System.out.println(max(2, 5));   // expect 5
7     System.out.println(max(7, 1));   // expect 7
8 }
```

Key points: • Test small pieces first; • Use known expected outputs; • Edge cases matter

Debugging methods (IDE workflow)

- Use breakpoints inside the method
- *Step Into:* follow the call into the method
- *Step Over:* run the method without entering
- Watch parameters and local variables
- If a bug is inside a method, isolate it with a small test case

Best practices in method design

- Keep methods short, aim for one clear purpose
- Prefer returning values over printing from deep inside logic
- Choose descriptive names, avoid single-letter names except loops
- Avoid hidden side effects, be explicit about what changes
- Document assumptions: valid ranges, null handling, units

Documenting method contracts (Javadoc)

```
1 /**
2  * Computes the average of values[0..count-1].
3  * @param values array containing the values
4  * @param count number of valid elements to use
5  * @return the average, or 0.0 if count == 0
6  */
7 static double average(int[] values, int count) {
8     if (count == 0) return 0.0;
9     int sum = 0;
10     for (int i = 0; i < count; i++) sum += values[i];
11     return (double) sum / count;
12 }
```

Key points:

- **Explain intent and assumptions;**
- ***@param* and *@return* build a contract;**
- **Helps future you and teammates**

Bridge to Week 4: from methods to classes

This week (utility methods)

- static double average(int[] values, int count)
- static int calculateSum(int[] values)
- static int readIntInRange(Scanner in, ...)

Next week (classes + methods)

```
BankAccount
-------------------
- owner: String
- balance: double
-------------------
+ deposit(amount): void
+ withdraw(amount): boolean
+ getBalance(): double
```

Activity: design with data + behaviour

Mini design exercise (3 minutes):
Design a class for a simple Student record.

Decide:

- **3 fields (data) you need**
- **3 methods (behaviour) you need**

Write the names and types.

Wrap-up and next steps

- Refactoring is a skill, small safe steps are the goal
- Methods should have clear inputs, outputs, and responsibilities
- Test and debug methods in isolation
- *Next week:* classes, constructors, this, encapsulation
- Bring your refactored lab code, we will evolve it into objects